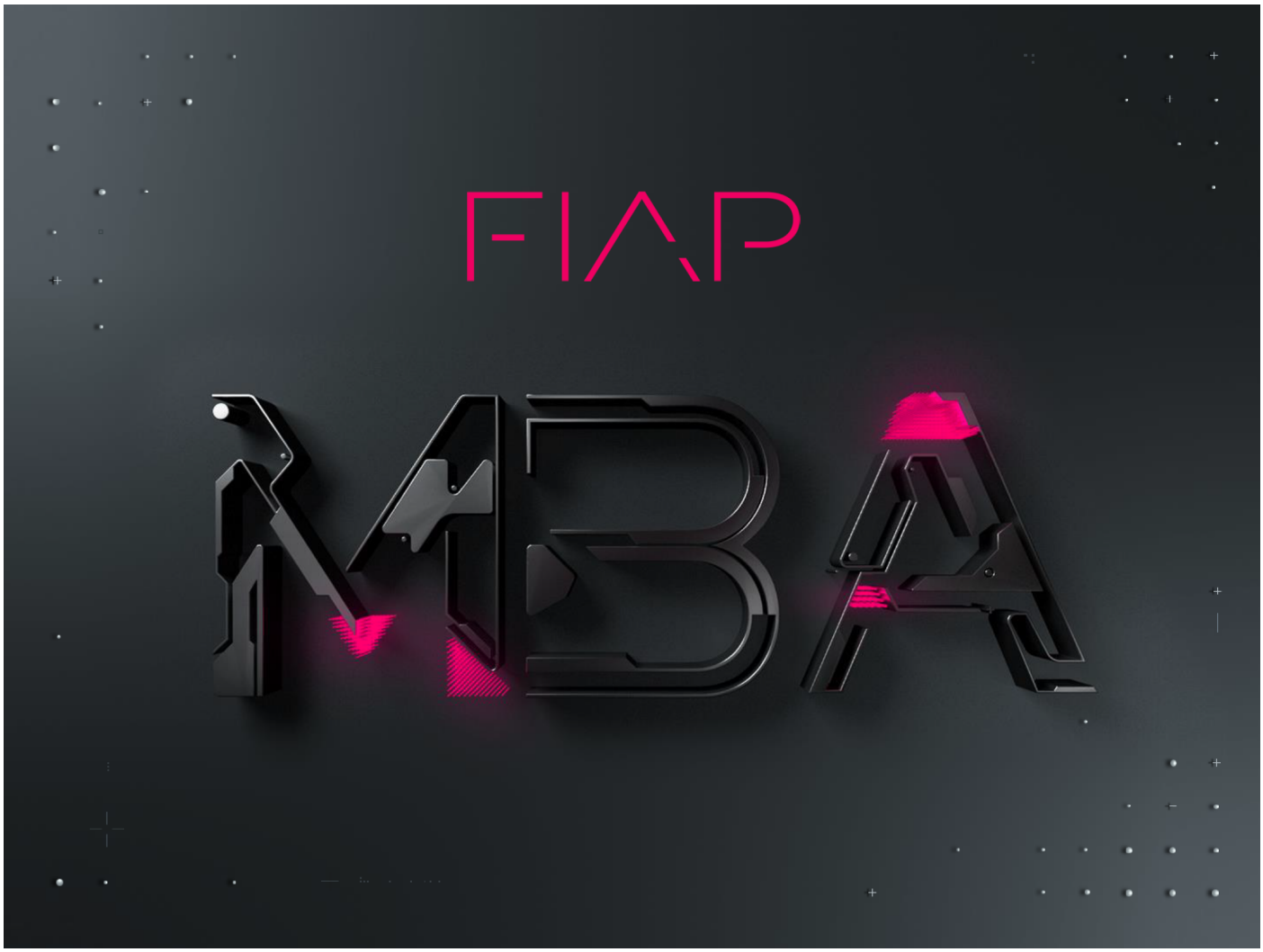

FIAP
MBA

Data Engineering Programming – Trabalho Final

Professor: **Marcelo Barbosa Pinto**

Objetivo

O objetivo deste trabalho é desenvolver um projeto pyspark utilizando os conhecimentos adquiridos durante o curso correspondentes aos assuntos abordados em sala de aula.

Escopo de negócio

Neste desafio você deve desenvolver um projeto Pyspark que resolva a seguinte questão:

A alta gestão da empresa deseja um relatório de pedidos de venda cujo pagamentos recusados (status=false) e que na avaliação de fraude foram classificados como legítimos (fraude=false).

O relatório deve ter os seguintes atributos:

1. Identificador do pedido (id pedido)
2. Estado (UF) onde o pedido foi feito
3. Forma de pagamento
4. Valor total do pedido
5. Data do pedido

O relatório deve compreender pedidos apenas do ano de 2025.

O relatório deve estar ordenado por estado (UF), forma de pagamento e data de criação do pedido.

O relatório deve ser gravado em formato parquet.

Entregáveis do trabalho

A resolução deste trabalho será considerada válida mediante a apresentação de evidências por imagens (prints de tela) dos respectivos códigos-fontes.

Os códigos-fontes do trabalho também devem ser disponibilizados em ****repositório público**** no Github.

Critérios gerais de avaliação

1. Todos os itens do ****escopo**** devem ser desenvolvidos neste trabalho;
2. Todos os itens dos ****critérios gerais**** e ****critérios específicos**** devem ser considerados;
3. Este trabalho deve ser entregue em documento único no formato ****PDF****;
4. Dúvidas sobre o escopo e execução devem ser esclarecidos em sala de aula ou através do email ****profmarcelo.barbosa@fiap.com.br****;
5. A entrega do trabalho só será considerada válida se feita através do ****Portal do Aluno****;
6. A nota final será divulgada através do ****Portal do Aluno****;
7. O trabalho deve ter capa contendo:
    a. nome da disciplina
    b. nome e RM de cada integrante do grupo;
8. Recomenda-se utilizar o ambiente de laboratório disponibilizado pelo professor.
9. Limitar a coleta de evidências ao máximo de 20 linhas.
10. As evidências coletadas devem estar em fonte e tamanho razoáveis para visualização e avaliação do professor.
11. Incluir por extenso no documento o link para o repositório público no Github.

    Exemplo:

    Em vez de MeuProjeto,

    coloque o link por extenso: https://github.com/meuusuario/meu-projeto .

12. A entrega será considerada ****inválida**** se entregue no formato de Python Notebook (.ipynb)
13. O projeto deve se manter agnóstico à plataforma onde foi desenvolvido.
14. Não basta a entrega de códigos-fontes, o projeto deve ser entregue com capacidade de execução.
15. Todos os detalhes específicos para execução do projeto devem ser descritos em um arquivo README.md presente no mesmo repositório do código-fonte.

Critérios específicos de avaliação

Seu projeto deve contemplar os seguintes requisitos:

1. ****Schemas explícitos****

   TODOS os dataframes devem ter seus schemas explicitamente definidos (sem inferência)

2. ****Orientação a objetos****
   TODOS os componentes do projeto devem ser encapsulados em CLASSES.

3. ****Injeção de Dependências****

- UTILIZAR o `main.py` como Aggregation Root
- INSTANCIAR todas as dependências no fluxo principal em `main.py`
- INJETAR as dependências via aggregation root
- As seguintes classes serão avaliadas como dependência:
  - Classes de configuração
  - Classes de gerenciamento de sessão spark
  - Classes de leitura e escrita de dados
  - Classes de lógica de negócios
  - Classes de orquestração do pipeline

4. ****Configurações centralizadas****

- DEFINIR um pacote de configurações
- DEFINIR pelo menos UMA classe de configuração
- UTILIZAR a configuração no fluxo principal

5. ****Sessão Spark****

- DEFINIR um pacote de gerenciamento da sessão spark
- CRIAR uma classe de gerenciamento de sessão spark
- UTILIZAR a sessão spark no fluxo principal

6. ****Leitura e Escrita de Dados (I/O)****

- DEFINIR pelo menos um pacote de leitura e escrita de dados
- CRIAR pelo menos uma classe de leitura e escrita de dados
- UTILIZAR os pacotes de leitura e escrita no fluxo principal

7. ****Lógica de Negócio****

- DEFINIR um pacote de lógica de negócios
- CRIAR pelo menos uma classe de lógica de negócios
- UTILIZAR o pacote de lógica de negócios no fluxo principal

8. ****Orquestração do pipeline****

- DEFINIR um pacote de orquestração do pipeline
- CRIAR pelo menos uma classe de orquestração do pipeline
- UTILIZAR o pacote de orquestração no fluxo principal

9. **Logging**

- IMPORTAR o pacote `logging` na classe de lógica de negócios.
- CONFIGURAR o logging
  Exemplo:

  ```
  logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')`
  ```

- UTILIZAR o logging para registro das etapas do pipeline.

10. **Tratamento de Erros**

- UTILIZAR a estrutura `try/catch` para tratamento de erros na classe de lógica de negócios.
- UTILIZAR logging para registro do erro capturado.

11. **Empacotamento da aplicação**

- CRIAR o arquivo `pyproject.toml`
- CRIAR o arquivo `requirements.txt`
- CRIAR o arquivo `README.md`
- CRIAR o arquivo `MANIFEST.in`

12. **Testes unitários**

- CRIAR pelo menos um teste unitário para a classe de lógica de negócios.
- O teste DEVE ser executado com sucesso.
- UTILIZAR o pacote `**pytest**`.

Material de apoio

Todo o material de apoio, instruções e conteúdo pedagógico pode ser encontrado no repositório https://github.com/infobarbosa/pyspark-poo .

Datasets

Dataset de Pagamentos

O dataset de pagamentos está disponível no seguinte repositório:

https://github.com/infobarbosa/dataset-json-pagamentos

Utilize os arquivos no caminho `**dataset-json-pagamentos/data/pagamentos**`.

As especificações do dataset (formato, estrutura de atributos, etc) estão disponíveis no próprio repositório.

Dataset de pedidos

O dataset de pedidos está disponível no seguinte repositório:

https://github.com/infobarbosa/datasets-csv-pedidos

Utilize os arquivos no caminho `**datasets-csv-pedidos/data/pedidos/**`.

As especificações do dataset (formato, estrutura de atributos, etc) estão disponíveis no próprio repositório.